

Examensarbete hos Kretslopp & Vatten, Open Source-IoT-Arkitektur

- Teknisk Rapport

Sammanfattning

I dagsläget arbetar Kretslopp och Vatten på ett utdaterat sätt när det kommer till monitorering av smarta, internetuppkopplade enheter och datahantering av vattenflödet i Göteborgs stad. Göteborg en av de städer som är delaktiga i EU-projektet “SCOREwater project” där målet är att främja hållbar vattenförvaltning i stadsområden.

Denna tekniska rapport innehåller detaljerad information om flertalet projekt där målbilden var att ha en fungerande open-source arkitektur inom Internet of Things för att säkert kunna provisionera, monitorera, hantera och visualisera data från smarta vattensensorer utsatta runt om i Göteborg samt lokalt i organisationens lokaler. Projektet hade som mål att demonstrera potentialen i open-source-lösningar för att stärka organisationen.

Projektet hade både framgångar och utmaningar, med en betydande mängd felsökning som behövde genomföras. För att denna typ av lösning ska vara möjlig och skalbar att utföra krävs en högre kompetensnivå inom organisationer och företag. Trots ansträngningar gick det inte att fullborda projektet inom tidsramen.

Skribenterna har skrivit respektive delar tillsammans men avskiljer sig vid kapitlen Diskussion och Slutsats, i denna version av dokumentet är dessa kapitlen skrivna av .

Lista över förkortningar och begrepp

Applikationsserver		En applikationsserver är en programvara som är utformad för att hantera och köra applikationer och tjänster för användning av flera användare på ett distribuerat nätverk. Inom IoT kan applikationsservern hantera applikationer som bearbetar och tolkar data från sensorer eller annan utrustning för att utföra uppgifter som automatisering av processer, analys eller visualisering av data.
CentOS8	Operativsystem	En Linux-distribution som bygger på den öppna källkoden i Red Hat Enterprise Linux (RHEL). Det är en stabilitetsorienterad och användarvänlig distribution som är populär inom servermiljöer. CentOS 8 erbjuder en robust och pålitlig plattform för att köra applikationer och hantera servrar.
CFSSL	Applikation	CloudFlare's SSL och är ett verktyg som används för att hantera X.509-certifikat och nycklar
Clusters		En grupp av servrar eller noder som samarbetar för att erbjuda en specifik tjänst eller funktionalitet.
Device Management	Enhetshantering	Enhetshantering innefattar olika strategier att tillhandahålla hårdvara på ett sätt som innebär att data kan vara säker, uppdaterad och hanterad på ett smidigt sätt.
Docker	Applikation	Docker är en plattform för att skapa och köra programvaruapplikationer i isolerade miljöer, kallade "containrar".
Docker-Compose	Applikation	Ett verktyg för Docker som används för att definiera och köra flera Docker-containers som en del av en applikation.

GDPR	General Data Protection Regulation	En dataskyddsförordning som antogs av Europeiska Unionen. Syftet med GDPR är att skydda privatlivet och skyddet av personuppgifter för EU-medborgare.
IoT	Internet Of Things	En teknik där fysiska enheter, som t.ex. sensorer och maskiner, är uppkopplade till internet och kan kommunicera med varandra. Detta möjliggör att data kan samlas in från en mängd olika enheter och sedan bearbetas och användas för att optimera processer, förbättra beslutsfattande och skapa nya möjligheter.
IoT-Gateway	En fysisk hårdvara	En IoT-gateway fungerar som en mellanhand mellan IoT-enheter och molntjänster eller andra system. Gatewayen samlar in data från de olika enheterna och bearbetar sedan informationen för att vidarebefordra den till ett moln eller andra system där data kan analyseras och användas för att optimera processer och fatta beslut.
Java 11 (OpenJDK)		Java 11, baserat på OpenJDK (Open Java Development Kit), är en version av programmeringsspråket Java och den tillhörande plattformen för att utveckla och köra Java-applikationer.
KoV	Kretslopp Och Vatten	En organisation för Göteborgs Stad.
LIA	Lärande Inom Arbete	En praktikperiod under yrkeshögskolans utbildningstid.
MD5-Authentication	Autentisering	En kryptografisk algoritm som används för att kryptera lösenordet som används vid autentisering. Genom att använda MD5-autentisering krypteras lösenordet som skickas mellan klienten och servern, vilket ökar

		säkerheten och förhindrar att lösenordet skickas som ren text.
MQTT-Protokoll	Kommunikationsprotokoll	MQTT (Message Queuing Telemetry Transport) är ett lättviktigt kommunikationsprotokoll som används för att skicka meddelanden mellan enheter i IoT (Internet of Things)-miljöer. Protokollet är utformat för att vara enkelt, pålitligt och energieffektivt.
Nätverksserver		En nätverksserver är en programvara som är utformad för att hantera kommunikation och dataöverföring mellan enheter och nätverk. Inom IoT kan nätverksservern hantera kommunikationen mellan enheter och en molnbaserad server eller gateway, samt hantera säkerhetsaspekter som kryptering av data och autentisering av enheter.
Open Source	Öppen källkod	Open Source är en programvarumodell där källkoden till en mjukvara är tillgänglig för allmänheten att använda, modifiera och distribuera fritt. Detta innebär att användare kan se hur mjukvaran fungerar, göra förbättringar och anpassningar efter sina egna behov, och dela med sig av sina ändringar tillbaka till gemenskapen.
PostgreSQL	Databashanterare	En kraftfull och öppen källkodsrelationell databashanterare (RDBMS). Det är en av de mest avancerade och populära databaserna som används för att lagra och hantera strukturerade data.
Proof of Concept	PoC	En kortfattad och experimentell implementering eller demonstration av en idé, produkt eller teknik för att bedöma dess genomförbarhet och funktionalitet. .

SCOREwater Project	Internationellt EU-projekt	Ett europeiskt forsknings- och innovationsprojekt som syftar till att främja hållbar vattenförvaltning i stadsområden.
TLS	Krypteringsprotokoll	Transport Layer Security är ett krypteringsprotokoll som används för att säkra kommunikationen över nätverk, särskilt på internet. Det skapar en säker och krypterad kanal mellan en klient och en server för att skydda data från avlyssning och manipulation. TLS används oftast för att säkra webbtrafik, inklusive HTTPS-kommunikation. Det ger autentisering, integritet och konfidentialitet för att säkerställa att endast avsedda parter kan läsa och förstå den utbytta informationen.
Visual Studio Code	Källkodseditor	En lättviktig och högt anpassningsbar källkodseditor utvecklad av Microsoft.
Web-Ui	Webbgränssnitt	Avser det grafiska användargränssnittet (GUI) som används för att interagera med webbapplikationer. Det är den visuella delen av en webbsida eller webbapplikation som användare ser och interagerar med via webbläsaren
X509 Certifikat	Autentisering	Ett X509-certifikat används för att autentisera och säkra kommunikationen mellan olika parter över ett nätverk. Det är en digital identitet som bekräftar ägandet av en viss offentlig nyckel och kan användas för att säkerställa att kommunikationen är privat och att den sker med rätt part.

Sammanfattning	1
Lista över förkortningar och begrepp	2
1. Inledning	7
1.1 Kretslopp och Vatten, Göteborgs Stad	7
1.2 IVL Svenska Miljöinstitutet	7
1.3 Bakgrund	8
1.4 Syfte	8
1.5 Problemformulering	9
1.6 Avgränsningar och fokus	9
1.7 Arbetssätt	9
2. Teori	12
2.1 ThingsBoard	12
2.2 ThingsStack	12
2.3 Docker	12
2.4 LoraWAN gateway - Kerlink IfemtoCell	13
2.5 Sensative Multi-Sensor	13
2.6 Turbinatorn	14
2.7 SCOREwater EU Project	15
2.8 Netmore	15
2.9 Flödesschema för enhet	16
3. Resultat	17
4. Diskussion	18
5. Slutsatser	18
5.1 Rekommendationer	19
6. Referenslista	19
7. Bilagor	21

1. Inledning

Vi har valt att skriva om vår LIA-period hos Kretslopp Och Vatten (KoV) samt IVL Svenska Miljöinstitutet och de projekt vi varit involverade i eftersom det eftertraktades av vår handledare att skriva om vårt arbete och att det saknas en viss kompetens inom organisationen och hos andra företag inom det.

1.1 Kretslopp och Vatten, Göteborgs Stad

Kretslopp & Vatten är en verksamhet inom Göteborgs stad som ansvarar för att hantera avfall, återvinning, vattenförsörjning och avloppshantering i staden. De arbetar för att främja hållbar och miljövänlig hantering av resurser och vatten i Göteborg. Kretslopp & Vatten erbjuder insamling och hantering av avfall, inklusive sophantering och återvinningstjänster. De ansvarar också för att säkerställa en trygg och hållbar vattenförsörjning samt för att hantera och rena avloppsvatten. Genom att främja cirkulär ekonomi och hållbara lösningar bidrar Kretslopp & Vatten till en ren och hållbar miljö i Göteborgs stad.

Vår handledare, som vi har haft under LIA-perioden, besitter rollen som IT-Arkitekt på KoV. Han ansvarar för att utveckla och underhålla den övergripande tekniska arkitekturen för IT-systemen och applikationerna. Det är ett viktigt samarbete mellan företagsledningen och andra IT-team för att förstå organisationens affärsbehov och utveckla en IT-strategi därefter som stöder dem. Han ansvarar också för att definiera tekniska standarder och riktlinjer som säkerställer att alla IT-lösningar och system är väl integrerade och följer företagets övergripande IT-strategi. Det kräver också att han har kunskap om de senaste teknikerna och trenderna inom IT-området för att kunna rekommendera lämpliga tekniklösningar och verktyg för organisationen.

1.2 IVL Svenska Miljöinstitutet

IVL Svenska Miljöinstitutet (IVL) är ett oberoende forskningsinstitut och konsultföretag inom miljö och hållbar utveckling. De arbetar med att främja en hållbar samhällsutveckling genom att bedriva forskning, utveckla lösningar och erbjuda expertis och rådgivning inom olika miljöområden. IVL samarbetar med företag, myndigheter och organisationer för att ta fram kunskap och innovationer som kan bidra till att minska miljöpåverkan och främja en cirkulär ekonomi. Institutet genomför även miljöutredningar och bedriver utbildning och informationsverksamhet för att öka medvetenheten och kunskapen om hållbarhet.

1.3 Bakgrund

Under lång tid har Kretslopp och Vatten arbetat på ett sätt som uppfattats utdaterat enligt våran handledare från vår föregående praktik. Vattnet i Göteborg har monitorerats sedan tidigare och än idag på så sätt att bilar åker runt med monterad gateway/mast och samlar in data från sensorer runt om i staden. Datan som kommer in är den totala mätningen av vatten som strömmat igenom sensorn sedan senaste gången det utfördes.

Våran uppfattning på plats var att det har länge diskuterats om huruvida man kan gå tillväga för att effektivt kunna få in mätningar av vatten med hjälp av ett mer modernt system, nämligen IoT-arkitekturen.

Våran uppgift var att bygga en IoT-arkitektur för provisionering, monitorering, underhållning och visualisering av enheterna utsatta i Göteborgs stad. Ett stort fokus låg också på att detta skulle konstrueras med hjälp av Open Source eftersom att köpa en färdig lösning kan leda till både höga kostnader och känslig kunddata Kretslopp och Vatten inte vill dela med sig av. Just nu har Kretslopp och Vatten ett avtal med företaget Netmore för hantering av deras smarta enheter.

Netmore är ett företag som specialiserar sig på IoT (Internet of Things) och M2M (Machine-to-Machine) kommunikationstjänster. De erbjuder anpassade lösningar för att ansluta och hantera IoT-enheter och data. Netmore fokuserar på att tillhandahålla robusta och pålitliga nätverk för att möjliggöra kommunikation och datautbyte mellan IoT-enheter och molnbaserade system. Företaget erbjuder också konsulttjänster och stöd för att hjälpa sina kunder att implementera och optimera sina IoT-lösningar.

1.4 Syfte

Syftet med denna rapport är att ge en djupare inblick i hur Kretslopp och Vatten arbetat mot en IoT-lösning med smart-devices för övervakning och visualisering av vattenflöden eller vattenvärden i Göteborgs stad. Syftet är också att redovisa vårt arbete hos dem samt att bevisa hur effektivt Open Source kan vara för företag och organisationer som vill arbeta med smarta enheter.

Studien visar användarvänliga lösningar som Kretslopp och Vatten kan använda för att ha en väl fungerande IoT-arkitektur. Resultaten av denna studie kommer att bidra till en bättre förståelse för hur Kretslopp och Vatten kan förbättra situationen de besitter just nu.

1.5 Problemformulering

Hur, med hjälp av en fungerande IoT-arkitektur, kan Kretslopp och Vatten effektivisera monitoreringen av vatten i Göteborg?

Vilka utmaningar har man stött på innan och vilka utmaningar har vi stött på under projektet?

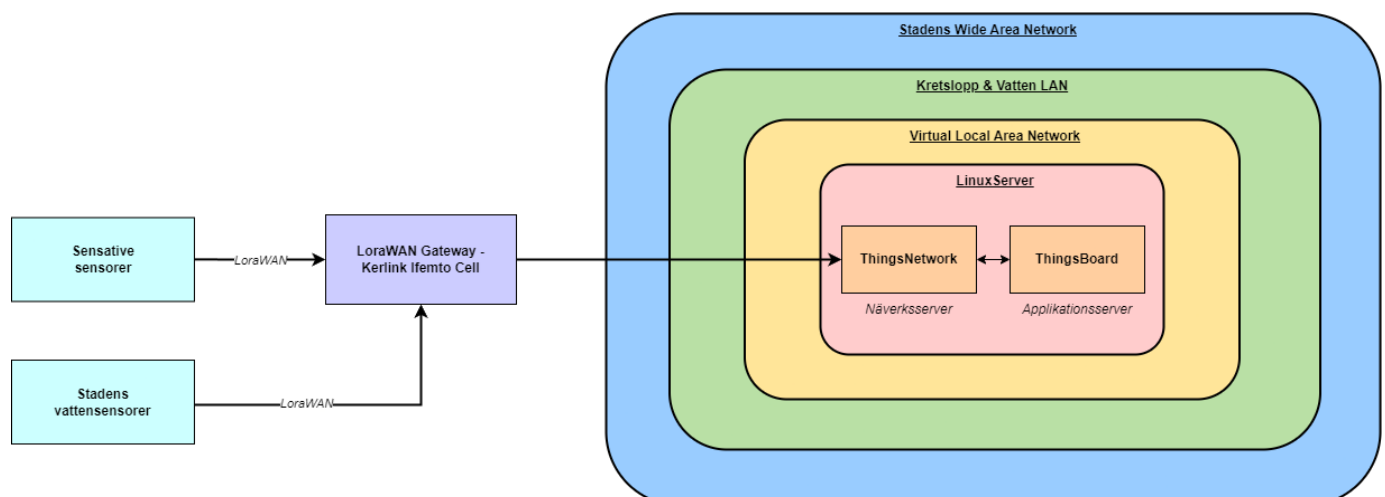
1.6 Avgränsningar och fokus

På grund av sekretessbelagd information från de olika projekten får vi praktikanter inte gå in på specifika detaljer vad vi har gjort, sett och varit delaktiga i under vår LIA-period. I stället kommer det att beskrivas i mer allmänna drag. Fokusen ligger endast på IoT-arkitekturen och vi kommer inte att analysera hur organisationen arbetar och utvecklas på andra fronter som inte är relaterade till IoT. Vi tänker endast analysera och diskutera kring Kretslopp och Vattens IoT-arkitektur samt IVL och kommer inte att undersöka hur andra organisationer eller företag arbetar.

1.7 Arbetssätt

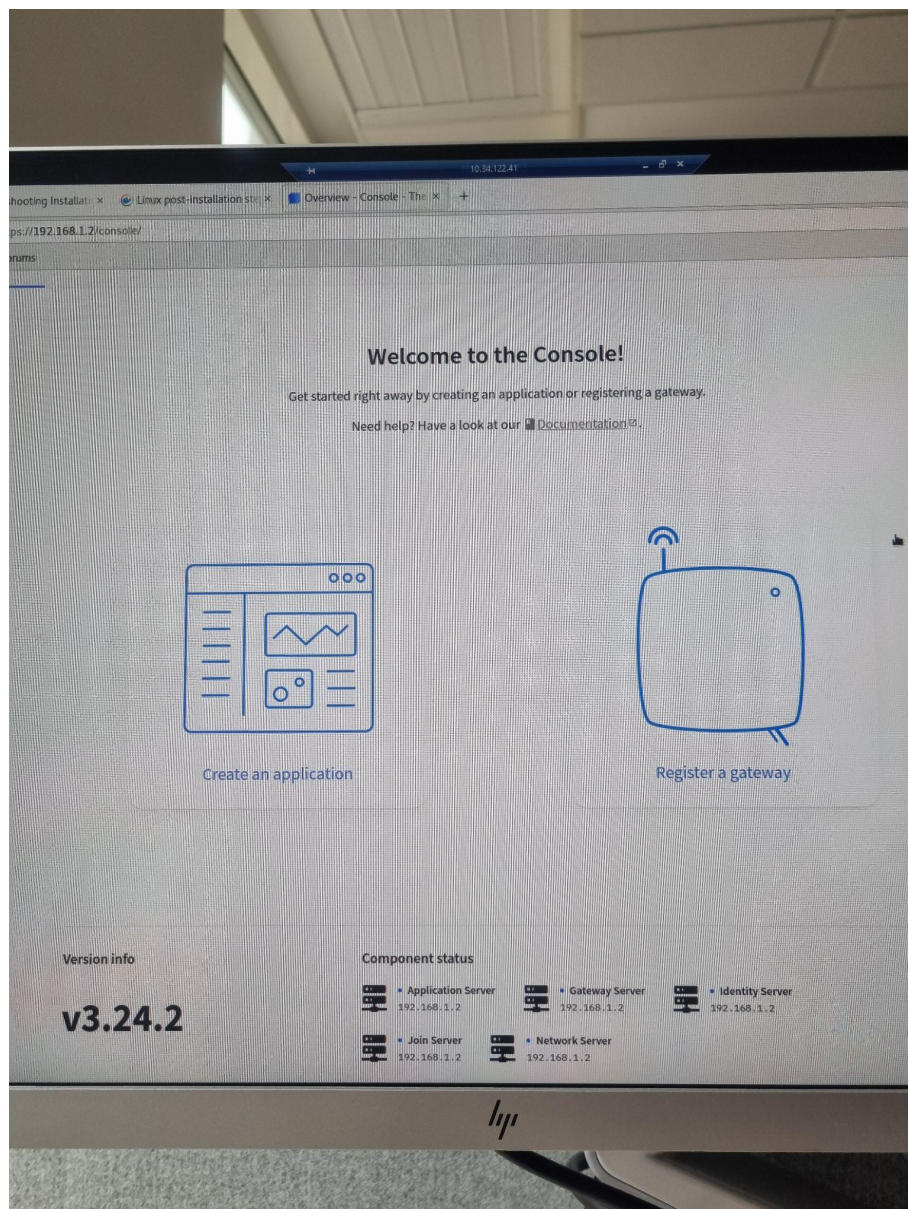
Vårt projektarbete gick ut på att vi skulle bygga en fungerande open-source IoT-arkitektur för KoV för att kunna demonstrera möjligheterna som finns inom IoT. Inom detta projekt har vi arbetat med att försöka hantera data från sensorer (Sensative IoT-Sensor, vattensensorer i Staden) med hjälp av en LoRaWAN gateway (Kerlink iFemtoCell), som vi arbetat med i en virtuell servermiljö. I servern har vi installerat och använt oss av Thingsboard, ThingsStack, och Docker.

Första steget i det hela projektet med KoV var att ha ett säkert nätverk att utföra projektet i. KoV bidrog då med ett virtuellt nätverk (VLAN) uppsatt som ett lokalt nätverk i deras lokaler. Kontoret bestod av flera våningar och vårt nätverk var endast tillgängligt att komma åt på en av våningarna. Vi fick sedan tillgång till en Linux-dator som i sin tur konfigurerades som både nätverks & applikationsserver.



Tack vare denna uppsättning kunde vi ansluta till vår Linux-dator (CentOS8) med hjälp av rätt kriterier och fjärrstyrning. Vi startade arbetet med en installation av ThingsStack som skulle agera som vår nätverksserver för att ha basen redo för våra enheter, i vårt fall till att börja med, sensitive-strips-sensorerna. För att lyckas med detta var vi tvungna att först generera ett X509 certifikat för att få en säker TLS-anslutning vid inlogg vid Web-Ui. Vi använde oss utav CFSSL från Cloudflare för att skapa egna certifikat med egna attributer som KoV tilldelade oss.

För att installera ThingStack korrekt använde vi oss utav Docker och dess starka egenskaper. Docker samt Docker-Compose installerades på vår Linux-dator och konfigurerades korrekt därefter. Docker använder sig av .yaml -filer som man i sin tur använder sig av variabler med specifika värden som vi bestämmer.



Se 7. Bilagor, sida 20, för bildserie av Docker-konfiguration.

De olika variablerna måste alltså stämma med till exempel vår faktiska, korrekta, lokala IP-adress på vår Linux-dator (värd) samt portar och annat som applikationerna behöver för att det ska bli korrekt. Vi bestämmer dessutom om vi ska använda certifikat eller inte. Av sekretessbelagd information kommer inte dessa certifikat gjorda för KoV att visas i denna dokumentation. Docker laddar sedan ner de applikationer vi vill använda oss av i denna så kallade "Docker-Container" och håller det igång så länge det är stabilt eller tills vi stänger ned det. Vi har använt oss av verktyget Visual Studio Code (VSC) för att skriva vår kod.

Väl inne i ThingsStack gränssnitt kunde vi lägga till vår LoraWAN-gateway (Ifemtocell Kerlink) med dess kriterier för att få en kommunikation.

Eftersom vi valde Docker som en tjänst för att hålla upp vår nätverksserver bestämde vi oss för att köra ThingsBoard som en service i själva operativsystemet CentOS8. ThingsBoard är beroende av Java11(OpenJDK), om det ska appliceras på detta vis, som i sin tur installerades.

Därefter hämtade vi ThingsBoard-paketet för att installera det på vår Linux-dator. Eftersom vi valde Professional Edition (PE) för ThingsBoard måste nyckeln tilläggas i det installerade ThingsBoard-paketets konfigurationsfil. Detta är för att de måste veta att vi har en giltig nyckel som vi betalar för.

ThingsBoard, utöver Java11(OpenJDK) är dessutom beroende av applikationen PostgreSQL. För att använda PostgreSQL installerades den senaste versionen. Vi skapade sedan den nödvändiga mappen och strukturen för att lagra databasfiler åt ThingsBoard samt skapade de grundläggande databasobjekten som behövs för att starta och använda PostgreSQL.

Därefter skapades den essentiella huvud-postgres-användaren som krävdes för våra syften. Vi konfigurerade PostgreSQL för att bestämma vilken typ av autentisering vi ville använda oss av. Valet föll på MD5-autentisering eftersom vi önskade att endast behöriga användare skulle kunna ansluta till PostgreSQL-databasen och utföra åtgärder. Genom att ändra inställningen för postgres-användaren till MD5-autentisering i konfigurationsfilen så kunde vi säkerställa att endast lösenord som var krypterade med MD5 kunde användas för att autentisera sig som postgres-användaren.



Sedan skapades en databas för att spara våra eventuella värden från sensorer samt felmeddelanden. Med det utfört var vi tvungna att konfigurera ThingsBoards konfigurationsfilen ännu en gång för att tillämpa vad just vår PostgreSQL-användare hade för namn samt lösenord.

ThingsBoard kan använda olika meddelandesystem/broker för att lagra meddelanden och kommunicera mellan ThingsBoard-tjänster. Standardimplementeringen är "In Memory"-kö,

vilket innebär att meddelandena lagras och hanteras endast i det lokala minnet. Valet föll på detta eftersom vi inte, till en början, använde många smarta enheter.. Denna implementering är användbar i utvecklings- eller Proof of Concept (PoC) miljöer där man behöver en enkel och snabb lösning för att testa och experimentera med ThingsBoard.

När väl allt detta blev utfört öppnade vi upp korrekt port (8080) då ThingsBoard User Interface (UI) använder det som standard och kunde därefter använda <http://localhost:8080/> för att nå ThingsBoard och dess tjänster.

2. Teori

2.1 ThingsBoard

Thingsboard är en open-source IoT-applikationsserver som hjälper användare att hantera och dra nytta av stora mängder data från anslutna enheter och sensorer. Det gör detta genom att tillhandahålla en mängd olika funktioner som gör det möjligt för användare att bygga och hantera skalbara IoT-applikationer.



2.2 ThingsStack

ThingsStack är en open source-nätverksserver för IoT som är speciellt utformad för att hantera kommunikationen mellan IoT-enheter och molnplattformar. Det är en viktig komponent i en LoRaWAN-baserad IoT-infrastruktur och tillåter användare att ansluta, registrera och hantera en stor mängd IoT-enheter som använder LoRaWAN-teknologi. Vilket våra olika sensorer har använt sig av. Kortfattat så är LoRaWAN en teknik för privata eller publika trådlösa nätverk för IoT. Det är en teknik för att överföra små mängder data över relativt långa sträckor.



2.3 Docker

Docker är en open source-plattform som används för att utveckla, distribuera och köra applikationer. Den tillhandahåller en containerbaserad virtualiseringsteknik som gör det möjligt för utvecklare att isolera och paketera applikationer och deras beroenden i en portabel och körbar enhet, kallad en Docker-container.



2.4 LoraWAN gateway - Kerlink IfemtoCell

Vår fysiska hårdvaru-LoraWAN gateway som vi använde var en Kerlink IfemtoCell. Den fungerar som en bro mellan IoT-sensorer och molnplattformar. I LoRaWAN-nätverk, som använder sig av LPWAN-teknologi, fungerar gatewayen som en central enhet som tar emot data från sensorerna och skickar den vidare till en molnplattform.



Gatewayen kan också skicka kommandon till sensorerna för att styra eller konfigurera dem. I grund och botten fungerar gatewayen som en viktig länk i IoT-nätverket som gör det möjligt att ansluta och integrera IoT-enheter och data i molntjänster och applikationer. Kerlink Wirnet iFemtoCell är en LoRaWAN-inomhusgateway som är idealisk för att ge täckning i byggnader, parkeringsgarage eller andra inomhusområden. Detta system använder sig av en integrerad antenn för att ge täckning över ett område på upp till 50 000 kvadratmeter.

Wirnet iFemtoCell stöder LoRaWAN-protokollet och ger hög prestanda även i krävande miljöer, till exempel källare eller hisschakt. Gatewayen har också enkel konfiguration och installation samt anslutning till molntjänster för fjärrhantering. Wirnet iFemtoCell är en kostnadseffektiv lösning för att integrera och ansluta IoT-enheter i ett LoRaWAN-nätverk och ger enkel anslutning och kommunikation med IoT-sensorer och andra enheter i smarta byggnader eller städer.

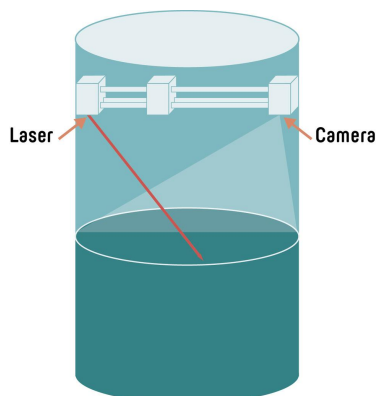
2.5 Sensative Multi-Sensor

De sensorer vi har arbetat med i projektet är Sensatives Multi-Sensor +Comfort för temperatur och luftfuktighet. Sensative tillverkar tunna, batteridrivna IoT-sensorer som kallas Strips, speciellt utformade för LoRaWAN-nätverk. Dessa sensorer är endast 3 mm tunna och kan enkelt monteras på nästan vilken yta som helst, både inomhus och utomhus. Strips-sensorerna kan mäta temperatur, fuktighet, ljus och rörelse, och de har också en inbyggd accelerometer för att detektera stötar eller vibrationer. Sensorerna har lång batteritid, upp till 10 år, och kan enkelt integreras med andra LoRaWAN-nätverk och IoT-plattformar. Strips-sensorerna är idealiska för att övervaka och optimera energiförbrukning, temperatur och luftfuktighet i byggnader, förbättra trafikflöden eller säkerhet, och för att övervaka miljöförhållanden inom jordbruk och livsmedelsindustri.



2.6 Turbinatorn

IVL har haft under en längre period arbetat med ett annat projekt vid namn “Turbinator”.



Sensorn skickar en laserstråle ner i vattnet, tar en bild av var ljuset träffar ytan och använder en AI-algoritm för att beräkna vattnets grumlighet baserat på bilden.

För att utveckla algoritmen har IVL samlat tusentals bilder tillsammans med referensmätningar. Bilderna används i träningsprocessen av algoritmen.

Genom att bearbeta informationen på både sensornivå och systemnivå med AI skapas en uppskattning av belastningen eller avvikelserna i nätverket över tid, vilket ger insikter som behövs för förebyggande underhåll av systemet.

Bilden laddas sedan upp med hjälp av MQTT-protokoll. Tanken är att AI:n ska lära sig när det är för höga värden för respektive brunn, där dessa smarta sensorer sitter.

På grund av sekretessbelagd information kan vi inte berätta på en detaljerad nivå exakt hur systemet och mjukvaran är utvecklat. Det är inte heller något vi har fått vara delaktiga i.

Vi har istället fått vara delaktiga i de första installationerna av “Turbinatorer”.

Bildsekvens. Installation av “Turbinator” vid flertalet brunnar. Se 7. Bilagor för mer bilder.



2.7 SCOREwater EU Project

SCOREwater EU (Smart Control of the Water Value Chain in Europe) är ett europeiskt forsknings- och innovationsprojekt inom ramen för Horizon 2020-programmet. Projektet är inriktat på att utveckla och demonstrera innovativa lösningar för att hantera och optimera vattenförsörjning och vattenhantering i stadsmässiga områden.



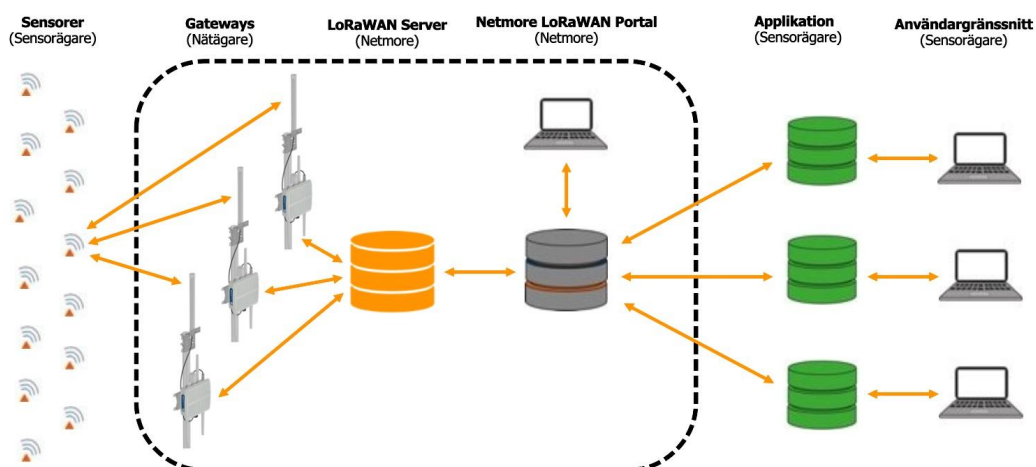
Målet med SCOREwater EU är att främja en smart och hållbar hantering av vattenresurser genom att integrera teknik, datadriven analys och beslutsstödssystem. Projektet fokuserar på att skapa smarta lösningar och verktyg som kan hjälpa till att förbättra effektiviteten, resurseffektiviteten och hållbarheten inom vattenförsörjning, avloppsrening och återanvändning av vatten. SCOREwater EU involverar ett partnerskap av akademiska institutioner, forskningsorganisationer, teknikföretag, vattenmyndigheter och kommuner från olika länder i Europa. Genom samarbete och kunskapsutbyte syftar projektet till att skapa innovativa lösningar och sprida bästa praxis för att förbättra vattenförvaltningen och bidra till en hållbar framtid. SCOREwater EU strävar efter att främja digitalisering, användning av sensorer och mätinstrument, dataanalys, modellering och artificiell intelligens för att optimera vattenförsörjningen och säkerställa en hållbar och effektiv hantering av vattenresurser i Europa.

Eftersom Göteborgs Stad är delaktig i SCOREwater EU finns det tydliga krav och mål som måste utföras i organisationen. Därför har man påbörjat ett arbete kring Internet of Things då man insett hur effektivt det kan vara.

2.8 Netmore

Netmore LoRaWAN Portal

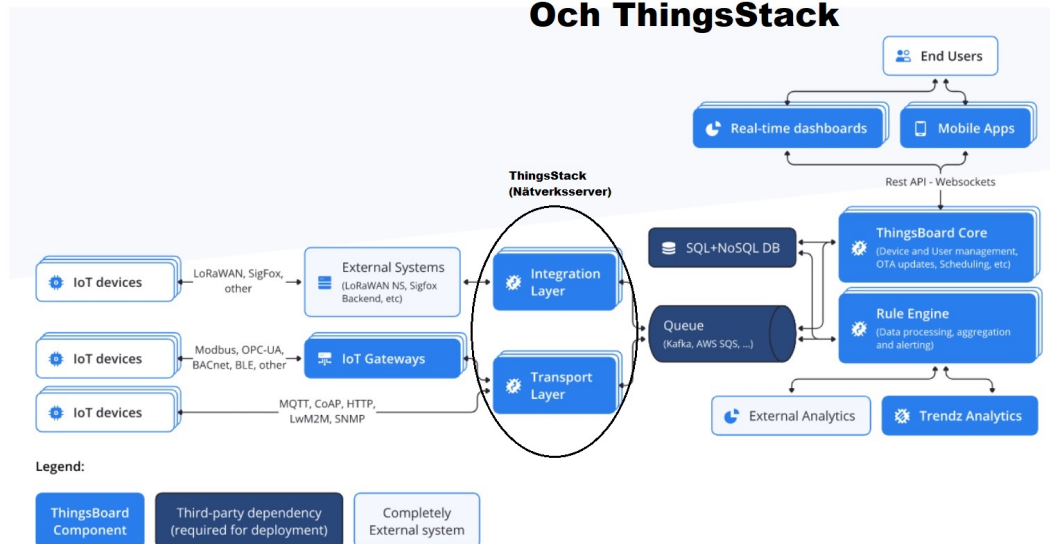
En webbaserad tjänst för samverkan mellan IoT Operatör, nätägare och sensorägare.



netmore

Som tidigare nämnt är Netmore deras nuvarande leverantör av mjukvara till deras smarta enheter men att man med tiden har insett problematik med visualisering, uppkoppling, hantering av enheter, samt kunddata. Av säkerhetsskäl insåg man att positionen av enheter och kundinformation inte ska hanteras av en tredje part. Eftersom organisationen är kommunal måste man ha GDPR i åtanke. Därför vill man istället nu satsa på egna lösningar inom området.

Framtidsvisionen med Thingsboard Och ThingsStack

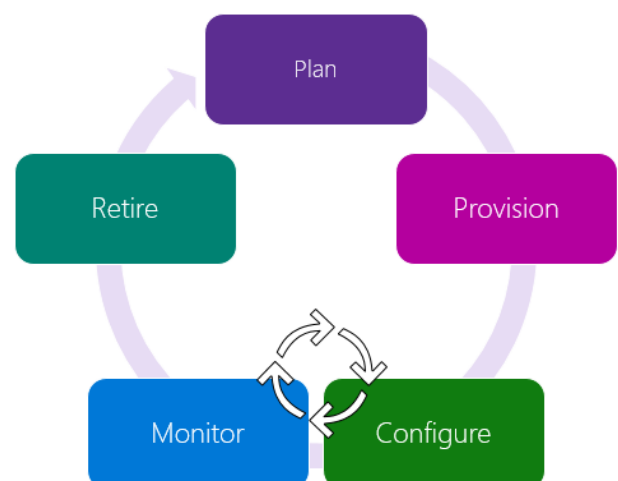


På bilden ovan syns IoT-arkitekturen som var målbilden för projektet med användningen av Thingsboard som applikationsserver och ThingsStack som nätverksserver.

2.9 Flödesschema för enhet

Det finns olika tabeller/flödesschema som beskriver hur livscykeln för en enhet ser ut, men de är alla liknande varandra. En viktig del mellan uppsättning av enhet och nedtagning är att kunna konfigurera och monitorera enheten för att förlänga dess livslängd, så att enheten inte behöver bytas ut ofta. Flödesschemat 2.1 nedan är ett exempel på ett sådant flödesschema tagen från en informativ sida om Microsoft Azure.

Figur för livscykel för smarta enheter enligt Microsoft.



3. Resultat

Vi stötte på många olika slags utmaningar under vår implementering av IoT-arkitekturen. Vid uppstart hade vi inte fått tillgång till vår VLAN som vi skulle få arbeta i och var tvungna att vänta in hårdvaran samt vårt VLAN från organisationen. Detta gjorde att vi blev försenade med att sätta upp vår IoT-arkitektur och kunde inte, helt enkelt, börja utföra det viktigaste arbetet förrän i början av mars. Tiden innan bestod dessutom av många vilseledande möten samt läsning från organisationen som inte har bidragit med någonting till projektet.

ThingsStack och LoraWAN-Gateway (Kerlink)

ThingsStack (nätverksservern) förblev stabil. Konfigurationen av vår LoraWAN-gateway (Kerlink) misslyckades. Väl inne i ThingsStack kunde vi tillämpa dess kriterier men vi fick felmeddelanden som "Wrong Cluster". Om en gateway visas i konsolen med statusen "Wrong Cluster" innebär det att denna gateway har konfigurerats med en Gateway Server-adress som konsolen bedömer inte tillhör den aktuella distributionen. Även fast vi lyckades ändra både cluster och testade olika ip-adresser på vår LoraWAN-gateway (Kerlink) fick vi samma status varje gång.

PostgreSQL felmeddelanden

Vi kunde inte få åtkomst till PostgreSQL eftersom det inte gick att logga in på databasen som vi skapat för ThingsBoard även fast vi använde de rätta inloggningsuppgifterna på huvudanvändare, och försök att uppdatera inloggningsinformationen i konfigureringsfilen misslyckades. Då ThingsBoard är beroende av just detta program så var det en av anledningarna till olika felmeddelanden. Detta testades även på våra personliga datorer utanför VLANet, utan några som helst problem med inlogg eller dylikt.

Problem med användningen av ThingsBoard Open Source samt integrering

Valet föll på Professional Edition (PE) av ThingsBoard för att integreringen med ThingsStack genom gratisversionen (Community Edition) inte är möjligt. ThingsStack och ThingsBoard är separata plattformar som har utvecklats och underhålls av olika team och organisationer. Detta innebär att de har olika arkitekturer, datastrukturer och API:er. För att integrera dessa två plattformar skulle det krävas betydande utvecklingsinsatser för att skapa en gemensam kommunikationskanal och datautbytesmekanism. Dessutom skulle vidareutveckling och underhåll av integrationen vara beroende av samarbetet mellan de två plattformarnas utvecklarcommunityn, vilket kan vara svårt att koordinera och upprätthålla över tid.

Problem med uppsättningen av Thingsboard PE (Professional Edition)

När vi väl försökte att starta igång Thingsboard PE på Linux-datorn så stötte vi på ett problem och tjänsten/service ville inte starta. Vi kom fram till att problemet, att driva Thingsboard som en tjänst/service genom Linux-datorn, var att Linux-datorn som skulle agera som värd (host) inte var tillräckligt kraftfull för att ha igång både ThingsStack (nätverksserver) och Thingsboard (applikationsserver) samtidigt då hårdvaran var avsevärt enkel. Dess processorkraft skred upp till 100% vid användning. Det resulterar i att

Thingsboard tvingar ner servicen/tjänsten automatiskt och försöker starta om sig själv i oändligheten.

I uppsättningen av Thingsboard PE genom Docker uppstod det ännu ett fel där nyckeln inte kunde hittas eller registreras av okända anledningar. Även detta testades på våra personliga datorer, som då inte var uppkopplade till projektets VLAN, utan några som helst felmeddelanden.

Slutligen ingen demo eller uppsatta enheter

Slutligen resulterade projektet i att inga av våra sensitive-sensorer eller Kretslopp & Vattens olika vattensensorer i tjänst blev uppkopplade eftersom applikationsservern inte kunde starta igång korrekt, därför blev det ingen demo uppsatt för organisationen.

4. Diskussion

Det finns säkerligen ett flertal sätt som Kretslopp och Vatten kan öka sin effektivisering av sin verksamhet med hjälp av smarta enheter, men det som var mest uppenbart tyckte jag var med de smarta vattenmätarna. Om de väl kan ta in datan mer effektivare än att åka runt med bilar och samla in datan så kan de få in den snabbare, samt att det sparar tid och pengar och värnar för miljön.

Det var ett svårt projekt att jobba med. Likt som Kretslopp och Vatten så saknade vår grupp kompetens inom IT-relaterade ämnen som nätverkshantering och programmering för att kunna utföra projektet på ett effektivt sätt inom projektets deadline.

Det behövdes planeras ordentligt, vilket var ett viktigt och svårare steg än förväntat. När vi sedan hade en plan så var det ändå oväntade hinder vid nästan alla steg i projektet som vi inte förväntade oss och att felsöka problemen var ett stort arbete som tog väldigt mycket tid.

5. Slutsatser

En följd av användning av smarta enheter som sensorerna, som Kretslopp och Vatten sätter ut, är att de inte behöver skicka ut arbetare för att kolla brunnarna på plats för att se ifall de behöver servas. Ett annat sätt som man kan göra arbetet smidigare är att man skulle kunna nyttja självreglerande sensorer som kan reglera vattenflöde eller liknande ifall det skulle behövas.

Men problemet med att börja använda smart devices är att det också behövs bra kompetens för att kunna skapa en bra IT-lösning. Man kan också hyra in system från andra företag, men då kan det bli problem gällande säkerheten om man arbetar med känslig data. Det kan också

vara så att utomstående företags lösning inte helt överensstämmer med det man vill åstadkomma inom IT.

Jag tror starkt att med tiden desto mer ett företag nyttjar rätt teknologier och kompetenser kan de öka produktivitet och kan lägga kraft på annat, och företaget kan växa mer än det skulle kunna ifall de inte använde sig av optimala teknologiska enheter.

5.1 Rekommendationer

Min rekommendation till företag eller organisationer som arbetar med data och teknologi är att undersöka och överväga ifall deras verksamhet kan utnyttja smarta enheter. Sedan bör verksamheten titta på de för- och nackdelar som det medför sig och ifall organisationen har den kompetens för att utveckla en egen lösning på enhetshantering, eller ifall det är bättre att hyra in utomstående kompetens.

6. Referenslista

Internet

Scorewater. Information och skrift om projektet

<https://www.scorewater.eu/cases/goteborg-en>

Kretslopp och Vatten - Allmän hemsida

<https://goteborg.se/wps/portal/start/kommun-och-politik/kommunens-organisation/forvaltning-ar-och-namnder/kretslopp-och-vatten>

IVL - Forskningsinstitutets hemsida.

<https://www.ivl.se/english/ivl.html>

IVL - Turbinatorn allmän information

<https://www.ivl.se/english/ivl/project/the-turbinator.html>

IVL - Allmän dokumentation om SCOREwater-project

<https://www.ivl.se/vart-erbjudande/forskning/vatten/scorewater-smart-hallbar-vattenhantering-i-storstader.html>

Microsoft Learn. Om device management.

<https://learn.microsoft.com/en-us/azure/iot-hub/iot-hub-device-management-overview>

ThingsStack. Allmän dokumentation.

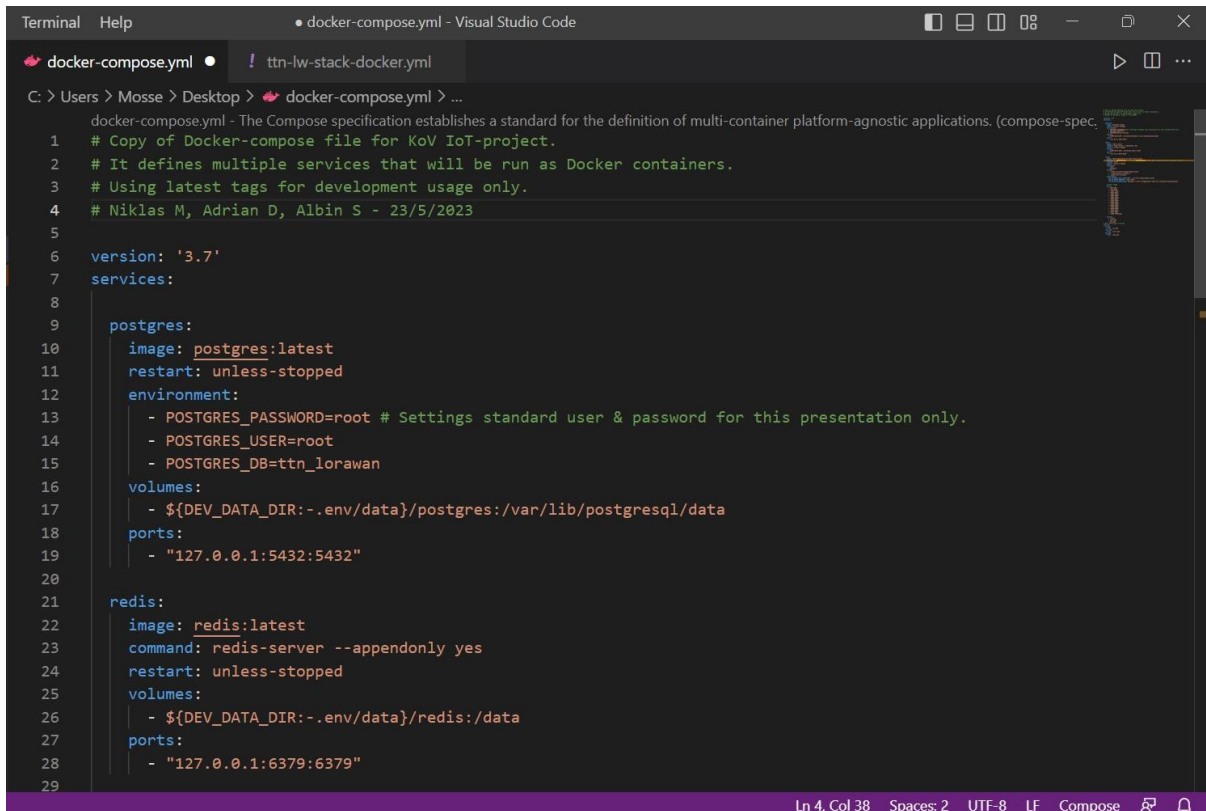
<https://www.thethingsindustries.com/docs/>

ThingsBoard. Allmän dokumentation.

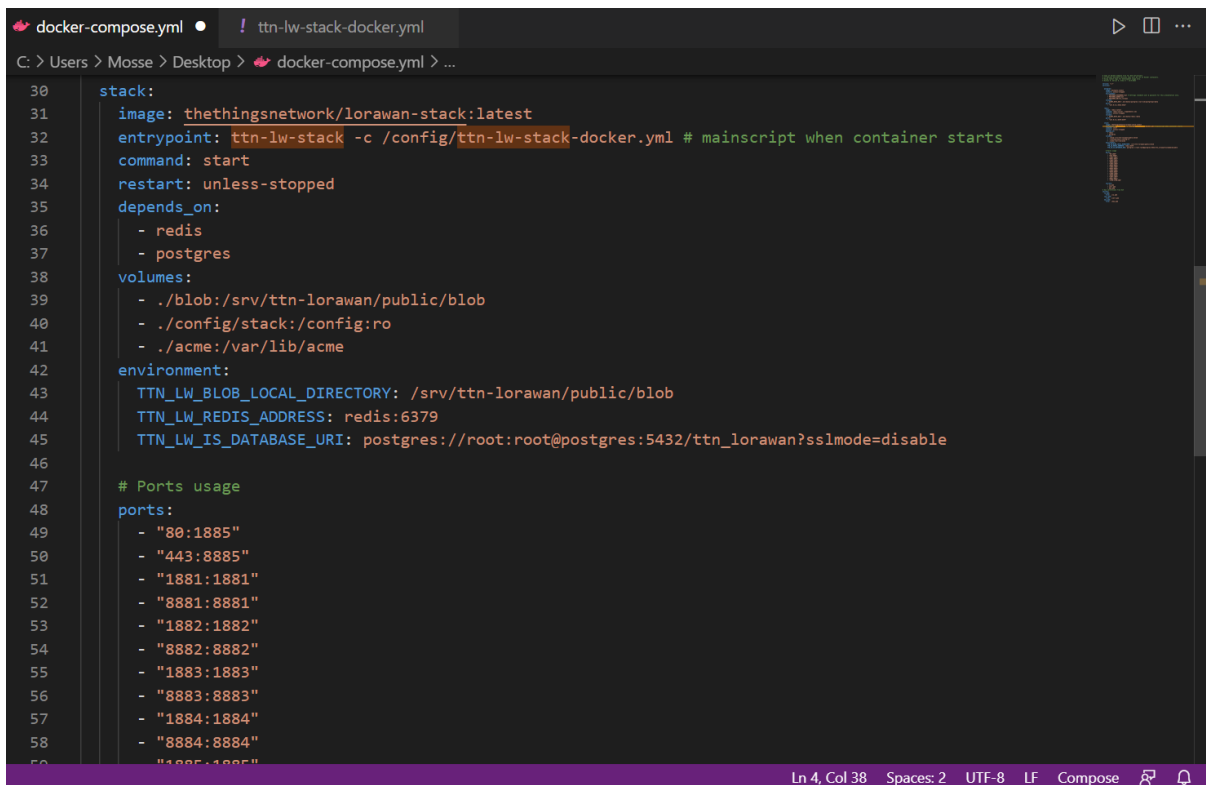
<https://thingsboard.io/>

7. Bilagor

Bildsekvens för Docker-kod samt skript. Docker-compose.yml samt ttn-lw-stack-docker.yml.



```
docker-compose.yml - Visual Studio Code
C: > Users > Mosse > Desktop > docker-compose.yml > ...
docker-compose.yml - The Compose specification establishes a standard for the definition of multi-container platform-agnostic applications. (compose-spec,
1 # Copy of Docker-compose file for KoV IoT-project.
2 # It defines multiple services that will be run as Docker containers.
3 # Using latest tags for development usage only.
4 # Niklas M, Adrian D, Albin S - 23/5/2023
5
6 version: '3.7'
7 services:
8
9   postgres:
10     image: postgres:latest
11     restart: unless-stopped
12     environment:
13       - POSTGRES_PASSWORD=root # Settings standard user & password for this presentation only.
14       - POSTGRES_USER=root
15       - POSTGRES_DB=ttn_lorawan
16     volumes:
17       - ${DEV_DATA_DIR:-.env/data}/postgres:/var/lib/postgresql/data
18     ports:
19       - "127.0.0.1:5432:5432"
20
21   redis:
22     image: redis:latest
23     command: redis-server --appendonly yes
24     restart: unless-stopped
25     volumes:
26       - ${DEV_DATA_DIR:-.env/data}/redis:/data
27     ports:
28       - "127.0.0.1:6379:6379"
29
```



```
docker-compose.yml • ! ttn-lw-stack-docker.yml
C: > Users > Mosse > Desktop > docker-compose.yml > ...
30 stack:
31   image: thethingsnetwork/lorawan-stack:latest
32   entrypoint: ttn-lw-stack -c /config/ttn-lw-stack-docker.yml # mainscript when container starts
33   command: start
34   restart: unless-stopped
35   depends_on:
36     - redis
37     - postgres
38   volumes:
39     - ./blob:/srv/ttn-lorawan/public/blob
40     - ./config/stack:/config:ro
41     - ./acme:/var/lib/acme
42   environment:
43     TTN_LW_BLOB_LOCAL_DIRECTORY: /srv/ttn-lorawan/public/blob
44     TTN_LW_REDIS_ADDRESS: redis:6379
45     TTN_LW_IS_DATABASE_URI: postgres://root:root@postgres:5432/ttn_lorawan?sslmode=disable
46
47   # Ports usage
48   ports:
49     - "80:1885"
50     - "443:8885"
51     - "1881:1881"
52     - "8881:8881"
53     - "1882:1882"
54     - "8882:8882"
55     - "1883:1883"
56     - "8883:8883"
57     - "1884:1884"
58     - "8884:8884"
59     - "1885:1885"

```

```
docker-compose.yml • ! ttn-lw-stack-docker.yml
C: > Users > Mosse > Desktop > docker-compose.yml > ...

59     - "1885:1885"
60     - "8885:8885"
61     - "1886:1886"
62     - "8886:8886"
63     - "1887:1887"
64     - "8887:8887"
65     - "1700:1700/udp"
66
67     secrets:
68     - ca.pem
69     - cert.pem
70     - key.pem
71 # Own certificate from KoV
72 secrets:
73   ca.pem:
74     file: ./ca.pem
75   cert.pem:
76     file: ./cert.pem
77   key.pem:
78     file: ./key.pem

Ln 4, Col 38  Spaces: 2  UTF-8  LF  Compose
```

```
docker-compose.yml • ! ttn-lw-stack-docker.yml X
C: > Users > Mosse > Desktop > ! ttn-lw-stack-docker.yml > ...

1  # Copy of Identity server configuration KoV IoT - Project
2  # Using 192.168.1.109 as local host example for this presentation only.
3  # Niklas M, Adrian D, Albin S - 23/5/2023
4
5  is:
6    email:
7      sender-name: "The Things Stack"
8      sender-address: "noreply@192.168.1.109"
9    network:
10     name: "The Things Stack"
11     console-url: "https://192.168.1.109/console"
12     identity-server-url: "https://192.168.1.109/oauth"
13
14    oauth:
15     ui:
16       canonical-url: "https://192.168.1.109/oauth"
17       is:
18         base-url: "https://192.168.1.109/api/v3"
19
20
21    http:
22     cookie:
23       block-key: ""
24       hash-key: ""
25     metrics:
26       password: "metrics"
27     pprof:
28       password: "pprof"
29
30 # Own certificate from KoV

Ln 3, Col 38  Spaces: 2  UTF-8  LF  YAML  No JSON Schema
```

```
docker-compose.yml • ! ttn-lw-stack-docker.yml X
C: > Users > Mosse > Desktop > ! ttn-lw-stack-docker.yml > ...
30 # Own certificate from KoV
31 source: file
32 root-ca: /run/secrets/ca.pem
33 certificate: /run/secrets/cert.pem
34 key: /run/secrets/key.pem
35
36 gs:
37   mqtt:
38     public-address: "192.168.1.109:1882"
39     public-tls-address: "192.168.1.109:8882"
40   mqtt-v2:
41     public-address: "192.168.1.109:1881"
42     public-tls-address: "192.168.1.109:8881"
43
44 gcs:
45   basic-station:
46     default:
47       lns-uri: "wss://192.168.1.109:8887"
48   the-things-gateway:
49     default:
50       mqtt-server: "mqtts://192.168.1.109:8881"
51
52 console:
53   ui:
54     canonical-url: "https://192.168.1.109/console"
55     account-url: "https://192.168.1.109/console"
56     is:
57       base-url: "https://192.168.1.109/api/v3"
58     gs:
59       base-url: "https://192.168.1.109/api/v3"
Ln 3, Col 38 Spaces: 2 UTF-8 LF YAML No JSON Schema
```

```
docker-compose.yml • ! ttn-lw-stack-docker.yml X
C: > Users > Mosse > Desktop > ! ttn-lw-stack-docker.yml > ...
59   base-url: "https://192.168.1.109/api/v3"
60   gcs:
61     base-url: "https://192.168.1.109/api/v3"
62     ns:
63       base-url: "https://192.168.1.109/api/v3"
64     as:
65       base-url: "https://192.168.1.109/api/v3"
66     js:
67       base-url: "https://192.168.1.109/api/v3"
68     qrg:
69       base-url: "https://192.168.1.109/api/v3"
70     edtc:
71       base-url: "https://192.168.1.109/api/v3"
72     dcs:
73       base-url: "https://192.168.1.109/api/v3"
74   oauth:
75     authorize-url: "https://192.168.1.109/oauth/authorize"
76     token-url: "https://192.168.1.109/oauth/token"
77     logout-url: "https://192.168.1.109/oauth/logout"
78     client-id: "console"
79     client-secret: "console"
80
81   as:
82     mqtt:
83       public-address: "192.168.1.109:1883"
84       public-tls-address: "192.168.1.109:8883"
85     webhooks:
86       downlink:
87         public-address: "192.168.1.109:1885/api/v3"
88
Ln 3, Col 38 Spaces: 2 UTF-8 LF YAML No JSON Schema
```



```
docker-compose.yml • ! ttn-lw-stack-docker.yml X
C: > Users > Mosse > Desktop > ! ttn-lw-stack-docker.yml > ...
88
89 dcs:
90   oauth:
91     authorize-url: "https://192.168.1.109/oauth/authorize"
92     token-url: "https://192.168.1.109/oauth/token"
93     logout-url: "https://192.168.1.109/oauth/logout"
94     client-id: "device-claiming"
95     client-secret: "device-claiming"
96   ui:
97     canonical-url: "https://192.168.1.109/claim"
98     as:
99       base-url: "https://192.168.1.109/api/v3"
100   dcs:
101     base-url: "https://192.168.1.109/api/v3"
102   is:
103     base-url: "https://192.168.1.109/api/v3"
104   ns:
105     base-url: "https://192.168.1.109/api/v3"
106
107 # This Dockerfile configures an instance of an Identity Server for the KoV IoT project.
108 # It includes information about email settings, network configurations,
109 # and various URLs for different components in the system.
110 # It also uses certificates and keys for TLS encryption and has settings for MQTT and webhooks addresses.
111 # The file specifies different base URLs for various subsystems used in the project.
```



Installation av Turbinator.